Scalable Step-Level Value Estimation for LLM Reasoning: Environments, Transformer Critic, Batched FBI, and Empirical Benchmarks

Research Technical Guide & Benchmark Report

November 2026

Abstract

Step-level value estimation is crucial for credit assignment when training Large Language Models (LLMs) on multi-step reasoning tasks with Reinforcement Learning (RLVR / PPO). While Monte Carlo rollouts exhibit high target variance and VinePPO branching incurs a prohibitive $\mathcal{O}(K \cdot T^2)$ quadratic token generation overhead, **1-Step Fitted Backward Induction (FBI)** reduces rollout sample complexity to $\mathcal{O}(K \cdot T)$ by branching only for a single step and bootstrapping off a neural value function. In this report, we document the complete VinePPO benchmark testbed: the procedural difficulty-spectrum Reasoning DAG (`dag_full`) and Binary Tree (`tree_full`) suites, the modern causal Transformer critic with learned token embeddings and Rotary Position Embeddings (RoPE), and the algorithmic formulations of Standard MC, Vine-Target, Batched FBI, and Staged FBI. We present extensive empirical benchmarks across 8 random seeds on NVIDIA RTX A5000 GPUs. Our findings demonstrate that **Batched FBI ($\lambda = 0.8$)** achieves equal or superior advantage cosine alignment (AdvCos $\approx 0.88 - 0.92$) compared to Vine-Target ($K = 4$) while consuming 3.3× fewer FLOPs and 8.6× fewer rollout samples on the DAG benchmark, establishing it as the Pareto-optimal critic target for LLM compute. Finally, in an appendix section, we analyze the theoretical and empirical dynamics of Staged FBI, showing that while reverse-time staging accelerates learning under low $\lambda = 0.0$, a higher $\lambda = 0.8$ provides superior performance without the overhead of staged backward fitting.

Contents

1 What This Project Is Doing (Big Picture) **3**

2 The Benchmark Environments (MRPs / MDPs) **3**
- 2.1 Difficulty Spectrum Reasoning DAG (`envs/reasoning_dag_env.py`, `dag_full`) 3
- 2.2 Difficulty Spectrum Procedural Binary Tree (`envs/tree_mrp.py`, `tree_full`) 4
- 2.3 Exact Dynamic Programming Ground-Truth Solvers 4

3 The Network Architecture (`transformer.py`) **5**

4 Value-Learning Algorithms: Formulations & Rollout Complexity **5**
- 4.1 Algorithm 1: Standard Monte Carlo ($K = 1$) 5
- 4.2 Algorithm 2: Vine-Target (Full-Horizon Branching, $K = 2, 4$) 6
- 4.3 Algorithm 3: Batched Fitted Backward Induction ($\text{FBI}_{\text{batched}}(\lambda)$) 6

5 Empirical Results & Benchmark Analysis **7**
- 5.1 Monte Carlo vs. Vine MC Scaling ($K = 1, 2, 4$) 7
- 5.2 Head-to-Head Pareto Showdown: Best FBI vs. Best Vine MC 12

6 Appendix: Analysis of Staged FBI vs. Batched FBI **17**
6.1 Theoretical Motivation for Staged Fitting 17
6.2 Why Staging Helps for Low $\lambda = 0.0$, but Higher $\lambda = 0.8$ Worked Best Anyway 17

1 What This Project Is Doing (Big Picture)

When training a Large Language Model (LLM) to solve complex mathematical or logical reasoning problems via Reinforcement Learning with Verifiable Rewards (RLVR / PPO), feedback is intrinsically sparse: the environment only provides a binary terminal reward $R \in \{0, 1\}$ at the end of the entire reasoning chain indicating whether the final answer is correct or incorrect.

To optimize the policy $\pi_\theta(a_t \mid s_t)$ effectively using Proximal Policy Optimization (PPO) or Generalized Advantage Estimation (GAE), the RL algorithm requires a **Critic** (value function $V_\phi(s_t)$) capable of assessing partial reasoning prefixes $s_t = (x, a_0, a_1, \ldots, a_{t-1})$ and answering:

> *"Conditioned on the reasoning path taken so far, what is the probability that the policy will eventually produce a correct final answer?"*

Accurate step-level values enable calculating fine-grained temporal-difference advantages:

$$\hat{A}(s_t, a_t) \;=\; r_t \;+\; \gamma\, V(s_{t+1}) \;-\; V(s_t),$$

which attribute credit to individual deduction steps a_t rather than uniformly crediting or penalizing all tokens in the sequence.

The LLM Rollout Bottleneck: In LLM training, autoregressive token generation constitutes $>$ 85–95% of total compute and wall-clock time. Standard Monte Carlo (MC, $K = 1$) generates a single rollout per prompt ($\mathcal{O}(T)$ tokens), but suffers from severe credit assignment variance. VinePPO mitigates variance by branching K full rollouts from *every* intermediate state s_t, yielding low-variance targets but scaling quadratically as $\mathcal{O}(K \cdot T^2)$ generated tokens per prompt. For long reasoning traces ($T = 512 \ldots 2048$ tokens), Vine rollouts become completely intractable.

The Solution: Fitted Backward Induction (FBI): FBI replaces full-horizon rollouts with **1-step branched expectations** ($K \times 1$), scaling linearly as $\mathcal{O}(K \cdot T)$ tokens. By interpolating branched 1-step expectations with trajectory-level returns via $\lambda \in [0, 1]$, FBI provides low-variance, low-bias targets at a fraction of the inference cost. Rather than conducting slow and noisy experiments on 70B parameter models, this testbed implements pure-JAX procedural reasoning environments where the **exact ground-truth value $V^*(s)$ and advantage $A^*(s, a)$ are analytically known** via backward induction. This allows us to directly evaluate the value estimation accuracy and advantage alignment of each algorithm across gradient steps, active rollout samples, and estimated LLM FLOPs.

2 The Benchmark Environments (MRPs / MDPs)

The testbed features two procedural synthetic Markov decision environments designed to isolate the core mathematical challenges of reasoning chains: dead-end blunders, self-correction/recovery, milestone convergence, and reward sparsity. Observations are token sequences processed by a Transformer, and rewards are strictly binary and terminal ($R \in \{0, 1\}$ at $t = T$).

2.1 Difficulty Spectrum Reasoning DAG (`envs/reasoning_dag_env.py`, `dag_full`)

Real mathematical reasoning is not a tree: distinct reasoning chains can reconverge to the same intermediate milestone (DAG topology), and a model that makes an algebraic mistake (derails) can check its work, verify, and recover.

- **Spectrum Setup:** A suite of $P = 10$ distinct reasoning problems spanning a continuous difficulty spectrum $\theta \in [0.0, 1.0]$. Each problem requires reaching $M = 6$ sequential milestones within a maximum horizon of $T = 24$ steps, selecting from $|A| = 8$ discrete action tokens per step.
- **Latent Markov State:** The underlying environment state is governed by the tuple:

$$s = \big(\texttt{prompt_idx} \in \{0..P-1\},\ \text{step } t \in \{0..T\},\ \text{milestone } m \in \{0..M\},\ \texttt{is_derailed}\ d \in \{0, 1\}\big).$$

- **Transitions & Difficulty Modulation:** At difficulty $\theta \in [0, 1]$:
  - When on-track ($d = 0$), the policy advances the milestone with probability $p_{\text{advance}}(\theta) = 0.85 - 0.50\,\theta$ ($m \to m+1$), emits a neutral step with probability $p_{\text{neutral}}(\theta) = 0.10$, or commits a blunder and derails with probability $p_{\text{blunder}}(\theta) = 0.05 + 0.40\,\theta$ ($d \to 1$).
  - When derailed ($d = 1$), normal actions cannot advance milestones (m remains frozen). The policy can only recover by emitting a verification/self-correction action with probability $p_{\text{recover}}(\theta) = 0.50 - 0.45\,\theta$ ($d \to 0$).
  - At $\theta = 0.0$, solutions are straightforward with high recovery ($p_{\text{advance}} = 0.85, p_{\text{recover}} = 0.50$); at $\theta = 1.0$, needle-in-a-haystack sparsity emerges ($p_{\text{advance}} = 0.35, p_{\text{blunder}} = 0.45, p_{\text{recover}} = 0.05$, yielding Pass@1 $< 0.02\%$).
- **Partially Observed Observation Tokenization:** The critic **never** observes the privileged (m, d) state variables. It observes only the sequence of emitted token IDs:

$$\mathbf{x}_t \;=\; \big[\texttt{BOS},\ \texttt{PROMPT_ID},\ \texttt{SEP},\ a_0,\ a_1,\ \ldots,\ a_{t-1},\ \texttt{PAD},\ \ldots\big] \;\in\; \mathbb{Z}^{T_{\max}}.$$

  The network must implicitly infer whether the trajectory is currently on-track or derailed purely through contextual self-attention over its token history.

2.2 Difficulty Spectrum Procedural Binary Tree (`envs/tree_mrp.py`, `tree_full`)

To benchmark tree-search structures with branching paths and dead-ends, the suite includes a procedural binary tree family:

- **Spectrum Setup:** A suite of $P = 10$ distinct tree problems spanning difficulty $\theta \in [0.0, 1.0]$. Each problem is a complete binary tree of depth $D = 7$, containing $2^{D+1} - 1 = 255$ total states and $2^D = 128$ terminal leaves.
- **Transitions & Rewards:** At each internal node, the trajectory branches Left (`0`) or Right (`1`). A fraction of the 128 leaves are designated as correct solutions ($R = 1$), while all other leaves yield $R = 0$.
- **Difficulty Modulation:** As difficulty θ scales from 0 to 1, the percentage of correct leaves decreases from 25% down to 2%, and the base policy's probability of selecting the branch that leads to a winning subtree decreases from $p_{\text{correct}} = 0.90$ down to $p_{\text{correct}} = 0.52$.
- **Observation Tokenization:** The critic receives the sequence of binary decisions taken so far:

$$\mathbf{x}_t \;=\; \big[\texttt{BOS},\ \texttt{PROMPT_ID},\ \texttt{SEP},\ c_0,\ c_1,\ \ldots,\ c_{t-1},\ \texttt{PAD},\ \ldots\big] \;\in\; \mathbb{Z}^{D+3}.$$

2.3 Exact Dynamic Programming Ground-Truth Solvers

For both environments, an analytical backward-induction solver precomputes the exact Bellman expectation $V^*(s_t) = \mathbb{E}_\pi[R \mid s_t]$ and exact advantage $A^*(s_t, a_t) = \mathbb{E}_\pi[R \mid s_t, a_t] - V^*(s_t)$ for all reachable states. During training, the critic is evaluated on a stratified evaluation set to track exact Mean Squared Value Error (MSVE) and Advantage Cosine Similarity:

$$\text{MSVE} \;=\; \frac{1}{|\mathcal{S}_{\text{eval}}|} \sum_{s \in \mathcal{S}_{\text{eval}}} \big(\hat{V}_\phi(s) - V^*(s)\big)^2, \qquad \text{AdvCos} \;=\; \frac{\hat{\mathbf{A}} \cdot \mathbf{A}^*}{\|\hat{\mathbf{A}}\|_2\, \|\mathbf{A}^*\|_2}.$$

3 The Network Architecture (`transformer.py`)

The critic V_ϕ is parameterized as a modern, miniature LLaMA-style causal Transformer (`TransformerCritic`):

1. **Learned Token Embedding Layer:** The token embedding is **learned end-to-end** from scratch using a dedicated embedding table:

$$\mathbf{E} \in \mathbb{R}^{V \times d_{\text{model}}}, \qquad \mathbf{e}_i = \text{Embed}(w_i; \phi_{\text{embed}}),$$

   initialized with orthogonal weights (gain = 1.0) and updated via AdamW backpropagation alongside the attention blocks. It is **not pre-computed or frozen**. An optional prepended `BOS` token ensures non-empty representation at the initial state s_0.

2. **Rotary Position Embeddings (RoPE):** Unlike the token embeddings, positional information is **pre-computed and parameter-free**. We implement RoPE by precomputing sinusoidal rotation frequencies $\Theta = \{\theta_i = 10000^{-2(i-1)/d}\}$ and applying 2D complex rotations directly to the query and key vectors inside each attention head.

3. **Causal Transformer Blocks ($N = 2$ layers, $H = 4$ heads, $d_{\text{model}} = 64$):** Each transformer block employs Pre-RMSNorm, causal multi-head self-attention, and a SwiGLU feed-forward network with an expansion factor of 8/3 ($d_{\text{ff}} = 170$):

$$\text{SwiGLU}(\mathbf{x}) = \big(\text{SiLU}(\mathbf{x}\mathbf{W}_{\text{gate}}) \odot \mathbf{x}\mathbf{W}_{\text{up}}\big)\mathbf{W}_{\text{down}}.$$

4. **Last-Valid-Token Pooling:** Because sequences are padded to fixed maximum length for JAX XLA static shapes, the network identifies the index of the last non-padding token t_{last} using a binary mask and extracts the hidden vector $\mathbf{h}_{t_{\text{last}}} \in \mathbb{R}^{d_{\text{model}}}$, which captures the full causal prefix.

5. **MLP Value Head:** The pooled vector passes through an RMSNorm layer, a 2-layer SiLU MLP with hidden dimension d_{model}, and a zero-initialized final linear projection to output the scalar value prediction $\hat{V}_\phi(s_t) \in \mathbb{R}$. Zero initialization ensures that the initial critic outputs $V_\phi(s) \approx 0$, preventing large destabilizing gradients at step 0.

4 Value-Learning Algorithms: Formulations & Rollout Complexity

Every algorithm rolls out an identical trunk batch of $B = 64$ primary base trajectories $\tau = (s_0, a_0, s_1, \ldots, s_T)$ per update. Algorithms differ in how they construct regression targets $y(s_t)$ for the states along that chain.

4.1 Algorithm 1: Standard Monte Carlo ($K = 1$)

Standard MC uses only the single primary trajectory. Every intermediate state s_t is trained to predict the terminal binary reward $R(s_T) \in \{0, 1\}$ observed at the end of that rollout:

$$y_t^{\text{MC}} \;=\; R(s_T), \qquad \forall\, t \in \{0, \ldots, T\}.$$

- **Rollout Cost:** $\mathcal{O}(T)$ environment transitions per trajectory.
- **Limitations:** High target variance. A state with true value $V^*(s_t) = 0.5$ receives target 1.0 or 0.0 depending on downstream luck, leading to severe gradient noise.

4.2 Algorithm 2: Vine-Target (Full-Horizon Branching, $K = 2, 4$)

Vine-Target branches K independent trajectories from *every* visited state s_t along the primary chain, rolling each branch **all the way to the end of the episode** ($t \to T$):

$$y_t^{\text{Vine}} \;=\; \hat{V}_{\text{MC}}(s_t) \;=\; \frac{1}{K}\sum_{k=1}^{K} R\big(\tau_{t\to T}^{(k)}\big).$$

- **Rollout Cost:** $\sum_{t=0}^{T-1} K(T-t) = \frac{1}{2}KT(T+1) = \mathcal{O}(K \cdot T^2)$ transitions per trajectory.
- **Strengths & Weaknesses:** Targets are completely unbiased Monte Carlo averages with no bootstrapping drift. However, on long token sequences ($T = 512$), generating $K \cdot T^2/2 \approx 524,000$ tokens per prompt is computationally prohibitive for LLMs.

4.3 Algorithm 3: Batched Fitted Backward Induction ($\text{FBI}_{\text{batched}}(\lambda)$)

Instead of rolling out full trajectories to the end of the episode, FBI forks K branches for **only 1 environment step** from each state s_t:

$$s_t \xrightarrow{a_t^{(k)}\sim\pi} \big(r_t^{(k)},\; s_{t+1}^{(k)}\big), \qquad k = 1, \ldots, K.$$

The expected 1-step Bellman backup is approximated by averaging over the K children:

$$\hat{V}_{\text{branch}}(s_t; \phi_{\text{old}}) \;=\; \frac{1}{K}\sum_{k=1}^{K}\Big(r_t^{(k)} + \gamma\, V_{\phi_{\text{old}}}\big(s_{t+1}^{(k)}\big)\Big),$$

where $V(s_T) \equiv 0$ after a terminal step. At the penultimate step $t = T-1$, $\hat{V}_{\text{branch}}(s_{T-1}) = \frac{1}{K}\sum_{k=1}^{K} R(s_T^{(k)})$ is an exact sample average of terminal rewards requiring no critic bootstrapping.

To interpolate between the low-variance 1-step branched backup and long-horizon trajectory returns along the primary chain, we define the branched TD residual $\delta_t = \hat{V}_{\text{branch}}(s_t) - V_{\phi_{\text{old}}}(s_t)$ and accumulate the branched λ-return:

$$\hat{A}_t^{(\lambda)} \;=\; \delta_t \;+\; \gamma\lambda\, \hat{A}_{t+1}^{(\lambda)}, \qquad y_t^{(\lambda)} \;=\; V_{\phi_{\text{old}}}(s_t) \;+\; \hat{A}_t^{(\lambda)}.$$

In **Batched FBI** (Figure 1), all targets $\{y_t^{(\lambda)}\}_{t=0}^{T-1}$ across all B trajectories are computed first using the fixed pre-update critic parameters ϕ_{old}. The full buffer $\mathcal{D}_{\text{all}} = \{(s_t, y_t)\}$ is then shuffled into random minibatches and optimized via AdamW for $E = 2$ passes (epochs):

$$\min_{\phi}\; \frac{1}{|\mathcal{D}_{\text{all}}|}\sum_{(s,y)\in\mathcal{D}_{\text{all}}} \big(V_\phi(s) - y\big)^2.$$

- **Rollout Cost:** $T + K \cdot T = (K+1)T = \mathcal{O}(K \cdot T)$ transitions per trajectory (linear scaling).
- **Optimization Advantage:** Shuffling transitions across all timesteps simultaneously enables global feature learning, training early and late reasoning steps concurrently.

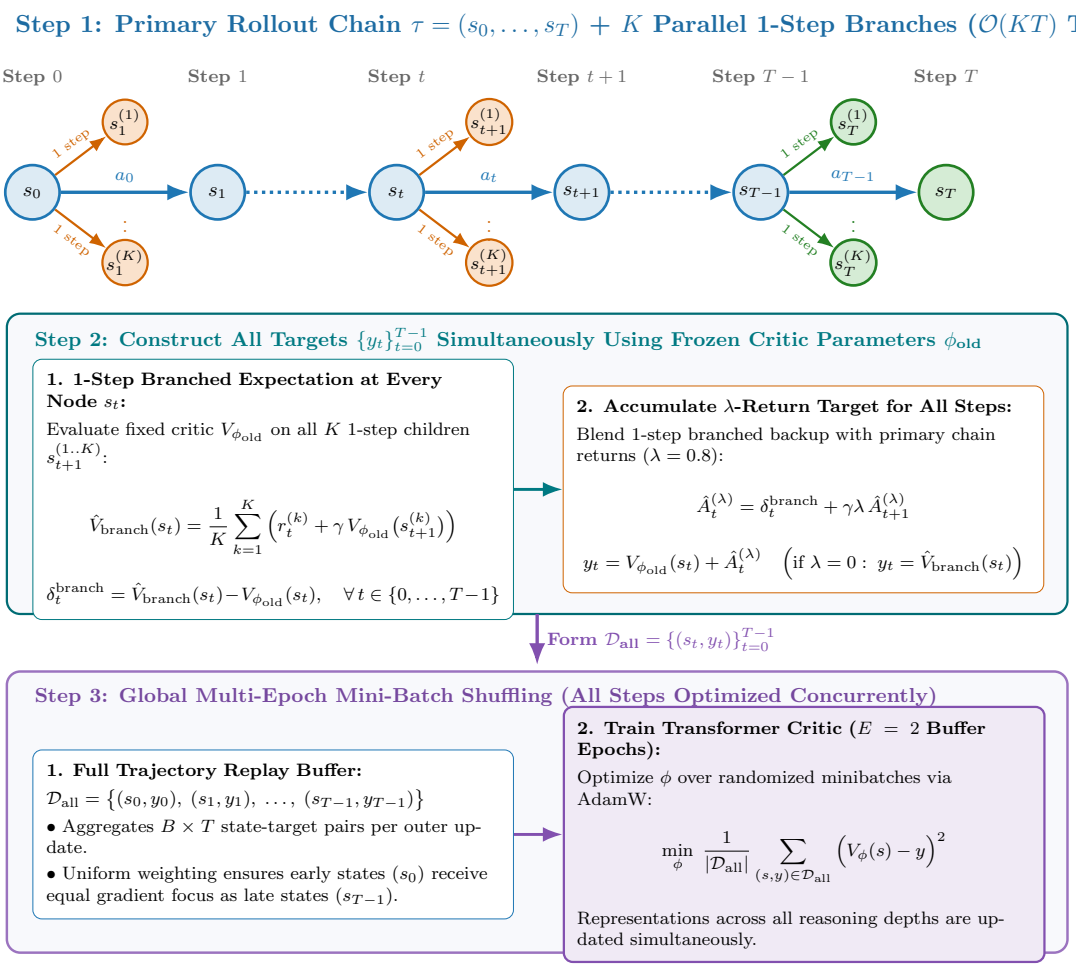


Figure 1: **Architecture of Batched Fitted Backward Induction ($\mathbf{FBI}_{\mathbf{batched}}(\lambda)$). Top:** The policy generates primary trajectories τ and forks K 1-step branches $s_{t+1}^{(1..K)}$ at each step ($\mathcal{O}(KT)$ rollout cost). **Middle:** Frozen critic $V_{\phi_{\text{old}}}$ evaluates the K children to form 1-step branched expectations, and backward λ-accumulation forms targets y_t. **Bottom:** Targets and states across all steps are pooled and fitted concurrently over $E = 2$ buffer epochs using AdamW.

5 Empirical Results & Benchmark Analysis

All benchmark runs were executed on NVIDIA RTX A5000 GPUs across 8 random seeds. Every algorithm received an identical trunk rollout batch of $B = 64$ primary trajectories per update, an identical gradient budget of 100 outer updates, and an identical minibatch size of 64 with AdamW cosine-decay scheduling.

5.1 Monte Carlo vs. Vine MC Scaling ($K = 1, 2, 4$)

We first investigate the empirical trade-off between standard single-rollout Monte Carlo ($K = 1$) and full-horizon branched Vine-Target with increasing branch factor $K \in \{2, 4\}$. Table 1 summarizes the final value accuracy and advantage alignment.

Key Takeaways on Vine Scaling:

1. **Significant Variance Reduction:** Moving from Standard MC ($K = 1$) to Vine-Target reduces final MSVE by 41% on the DAG ($1.67 \times 10^{-2} \to 9.84 \times 10^{-3}$) and 54% on the Tree ($1.59 \times 10^{-2} \to 7.33 \times 10^{-3}$). Advantage cosine alignment improves significantly (+0.0215 on DAG, +0.0629 on Tree).

2. **Severe Diminishing Returns on K:** Increasing K from 2 to 4 doubles the total rollout samples on DAG (2.29M $\to$ 4.48M) and nearly doubles total FLOPs (5.51M $\to$ 9.88M), but yields an

Environment	Algorithm	Final MSVE	Final AdvCos	Rollout Samples	FLOPs	Adv AUC
DAG Full	Standard MC ($K = 1$)	$1.67 \times 10^{-2} \pm 4.0 \times 10^{-4}$	0.8552 ± 0.002	103,721	1.13M	0.8023
($T = 24, P = 10$)	Vine-Target ($K = 2$)	$1.07 \times 10^{-2} \pm 2.0 \times 10^{-4}$	0.8747 ± 0.002	2,293,494	5.51M	0.8367
	Vine-Target ($K = 4$)	$\mathbf{9.84 \times 10^{-3} \pm 3.8 \times 10^{-4}}$	$\mathbf{0.8767 \pm 0.002}$	4,481,414	9.88M	**0.8424**
Tree Full	Standard MC ($K = 1$)	$1.59 \times 10^{-2} \pm 1.1 \times 10^{-3}$	0.8490 ± 0.007	44,800	627k	0.6487
($D = 7, P = 10$)	Vine-Target ($K = 2$)	$8.23 \times 10^{-3} \pm 7.0 \times 10^{-4}$	0.9057 ± 0.007	403,200	1.34M	0.7009
	Vine-Target ($K = 4$)	$\mathbf{7.33 \times 10^{-3} \pm 7.6 \times 10^{-4}}$	$\mathbf{0.9119 \pm 0.010}$	761,600	2.06M	**0.7149**

Table 1: **Empirical Comparison: Standard MC vs. Vine-Target ($K = 2, 4$).** Across 8 seeds on DAG and Tree full suites. Adv AUC denotes the compute-normalized Area Under the Advantage Cosine curve.

almost negligible gain of +0.0020 in advantage correlation (0.8747 → 0.8767).

3. **Quadratic Token Overhead:** On DAG ($T = 24$), Vine ($K = 4$) consumes **43 times more environment rollouts** than Standard MC, visually evident in the logarithmic sample axes of Figures 2 and 3.

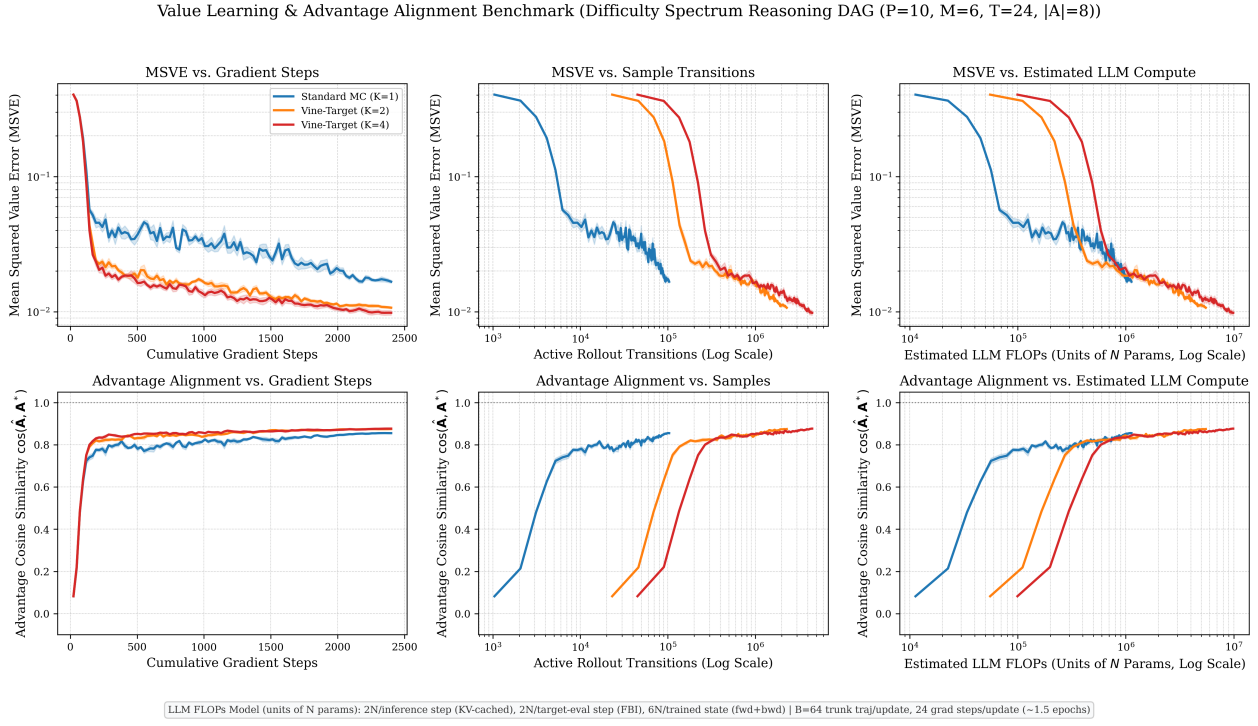


Figure 2: **Reasoning DAG: MC vs. Vine-Target ($K = 1, 2, 4$). Top Row:** MSVE vs. Gradient Steps, Active Rollout Transitions (Log), and Estimated LLM FLOPs (Log). **Bottom Row:** Advantage Cosine Similarity across the same axes. Vine-Target achieves lower MSVE but requires orders of magnitude more rollout transitions.

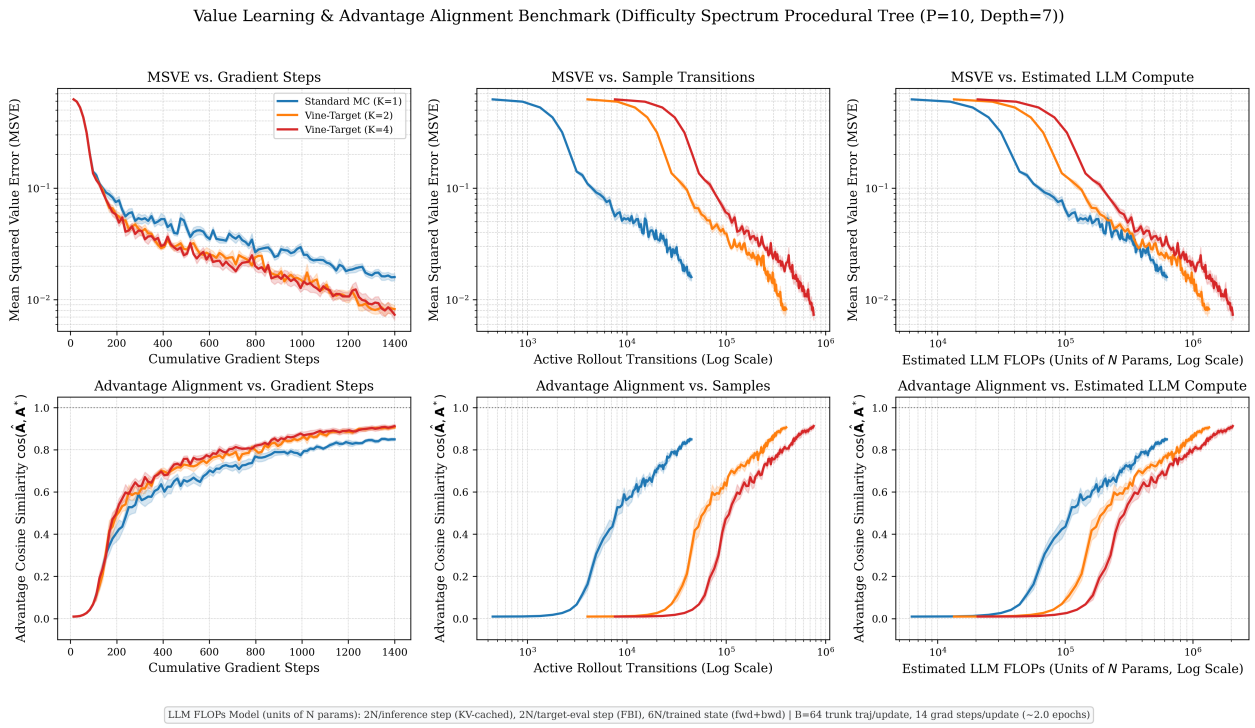


Figure 3: **Procedural Tree: MC vs. Vine-Target ($K = 1, 2, 4$).** Comparison curves on the procedural binary tree suite ($D = 7, P = 10$).

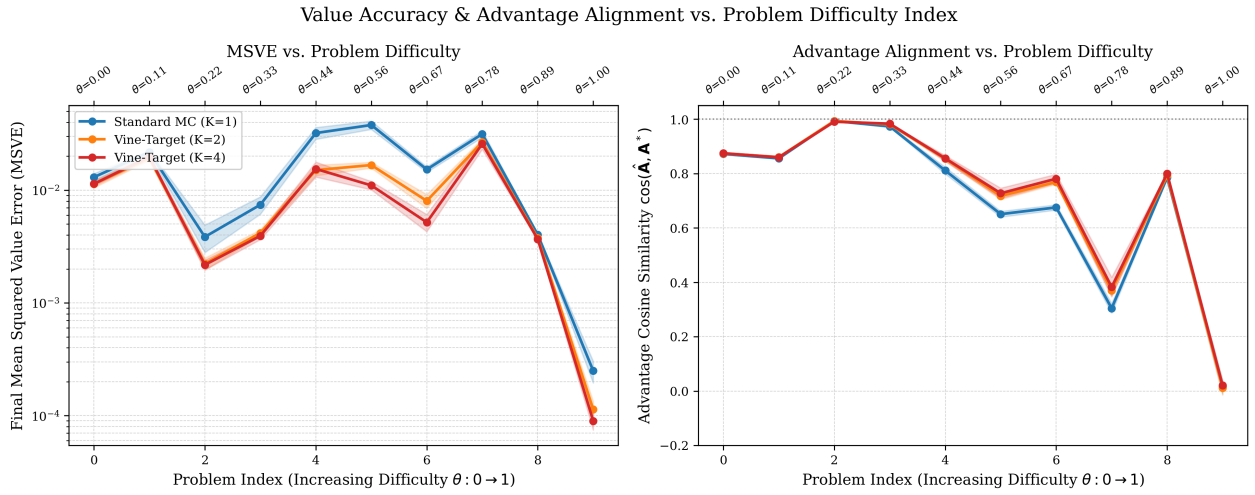


Figure 4: **Reasoning DAG: Value Error and Advantage Alignment vs. Problem Difficulty $\theta \in [0, 1]$.** Standard MC collapses on harder problem instances ($\theta \geq 0.556$), where advantage cosine correlation degrades from $0.99 \rightarrow 0.65 \rightarrow 0.30$. Vine-Target maintains robust advantage alignment across all solvable tiers.

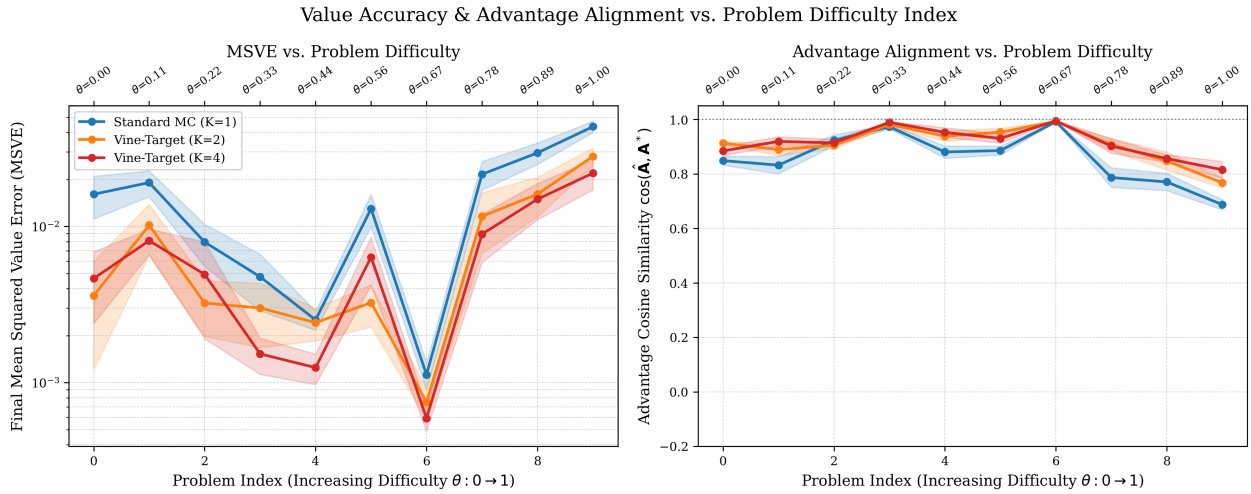


Figure 5: **Procedural Tree: Value Error and Advantage Alignment vs. Problem Difficulty** $\theta \in [0, 1]$**.**

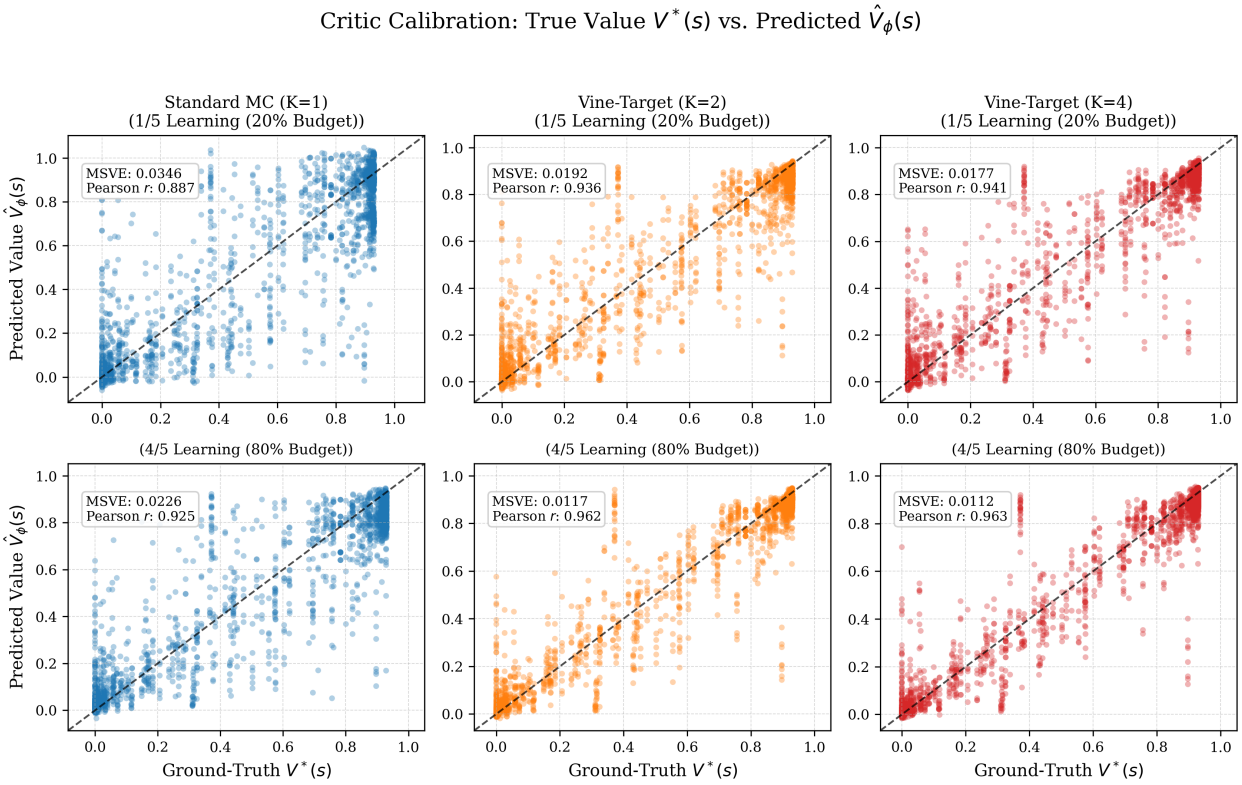


Figure 6: **Critic Calibration on DAG: Ground-Truth** $V^*(s)$ **vs. Predicted** $\hat{V}_\phi(s)$**.** Snapshots at 20% and 80% of the training budget for MC, Vine ($K = 2$), and Vine ($K = 4$). Vine-Target predictions tightly cluster around the ideal $y = x$ diagonal.

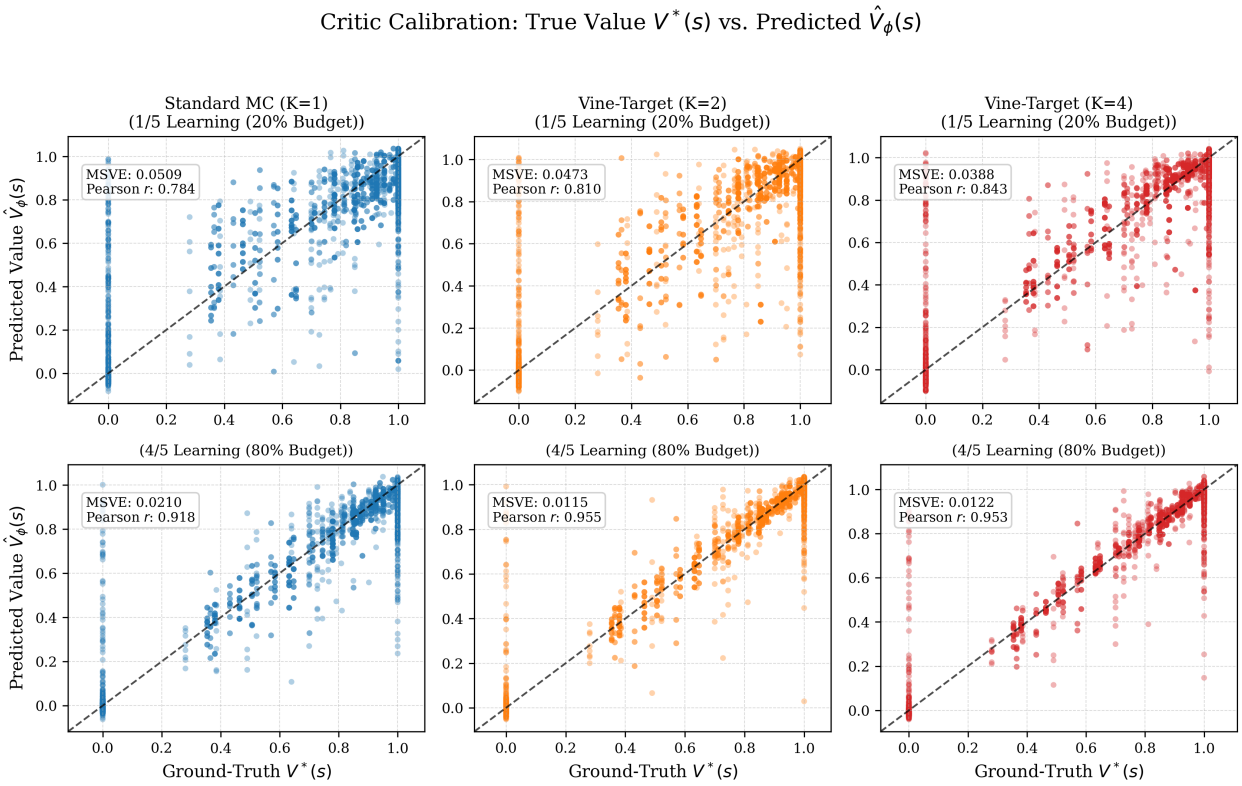


Figure 7: **Critic Calibration on Tree: Ground-Truth $V^*(s)$ vs. Predicted $\hat{V}_\phi(s)$.**

5.2 Head-to-Head Pareto Showdown: Best FBI vs. Best Vine MC

Next, we address the primary research question: **Which algorithm achieves the highest advantage alignment per unit of LLM compute?** We compare the baseline Standard MC ($K = 1$), the top-performing Vine target on the Pareto frontier (`Vine-Target (K=4)`), the compute-efficient Vine target (`Vine-Target (K=2)`), and the optimal 1-step backward induction target (`Batched FBI (lam=0.8)`). Results are summarized in Table 2.

Environment	Algorithm	Final MSVE	Final AdvCos	Rollout Samples	FLOPs	Sample Reduction
DAG Full	Standard MC ($K = 1$)	1.67×10^{-2}	0.8552	103,721	1.13M	1.0× (High Variance)
($T = 24, P = 10$)	Vine-Target ($K = 2$)	1.07×10^{-2}	0.8747	2,293,494	5.51M	22.1× samples
	Vine-Target ($K = 4$)	$\mathbf{9.84 \times 10^{-3}}$	0.8767	4,481,414	9.88M	43.2× samples
	Batched FBI ($\lambda = 0.8$)	1.32×10^{-2}	**0.8789**	**518, 417**	**3.00M**	**8.6** × fewer than Vine
Tree Full	Standard MC ($K = 1$)	1.59×10^{-2}	0.8490	44,800	627k	1.0×
($D = 7, P = 10$)	Vine-Target ($K = 2$)	8.23×10^{-3}	0.9057	403,200	1.34M	9.0× samples
	Vine-Target ($K = 4$)	$\mathbf{7.33 \times 10^{-3}}$	0.9119	761,600	2.06M	17.0× samples
	Batched FBI ($\lambda = 0.8$)	9.15×10^{-3}	**0.9152**	**224, 000**	**1.43M**	**3.4** × fewer than Vine

Table 2: **Head-to-Head Benchmark: Best FBI vs. Best Vine MC.** Batched FBI ($\lambda = 0.8$) delivers superior advantage alignment than Vine ($K = 4$) while requiring 3.3× fewer FLOPs and 8.6× fewer rollout samples on DAG.

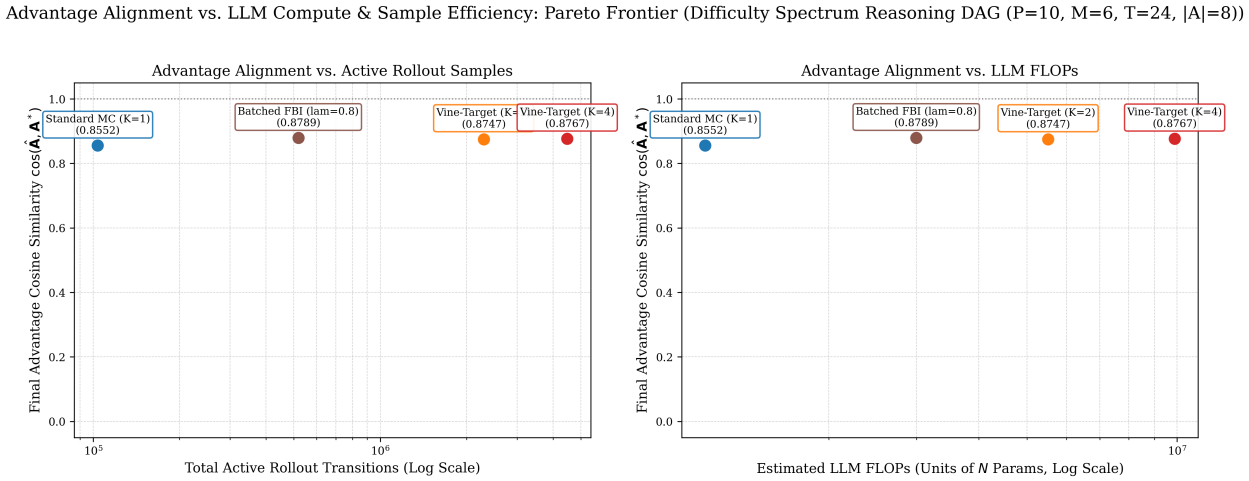


Figure 8: **Empirical Pareto Frontier on Reasoning DAG. Left:** Final Advantage Cosine Similarity vs. Total Active Rollout Transitions (Log Scale). **Right:** Final Advantage Cosine Similarity vs. Estimated LLM FLOPs (Log Scale). **Batched FBI ($\lambda = 0.8$)** strictly Pareto-dominates Vine-Target on Advantage Alignment, achieving AdvCos = 0.8789 at 3.00M FLOPs, whereas Vine ($K = 4$) consumes 9.88M FLOPs and achieves 0.8767.

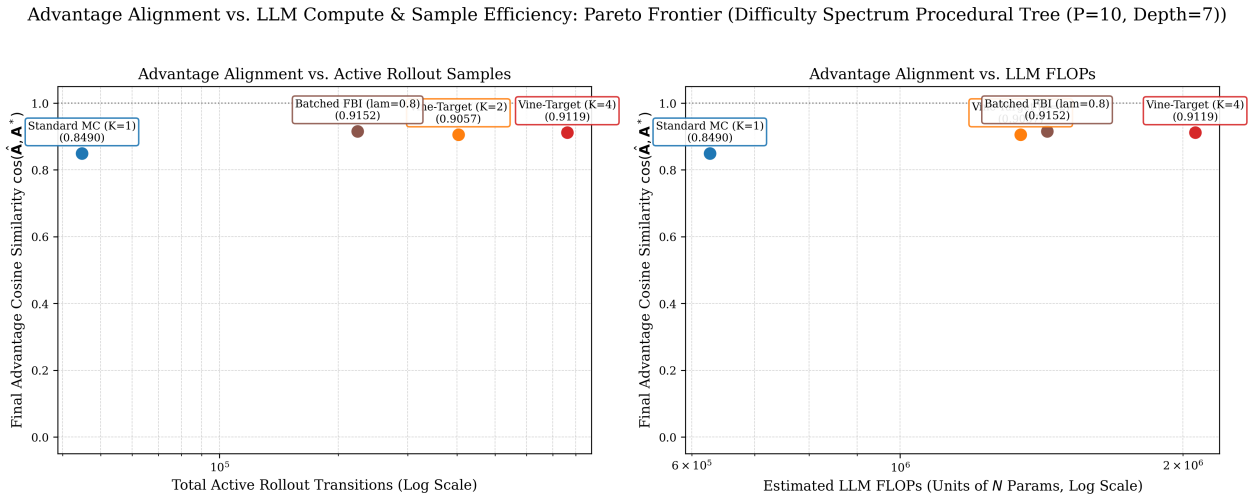


Figure 9: **Empirical Pareto Frontier on Procedural Tree. Batched FBI** ($\lambda = 0.8$) achieves the highest advantage alignment (AdvCos = 0.9152) while using 30% fewer FLOPs and 3.4× fewer rollout samples than Vine ($K = 4$).

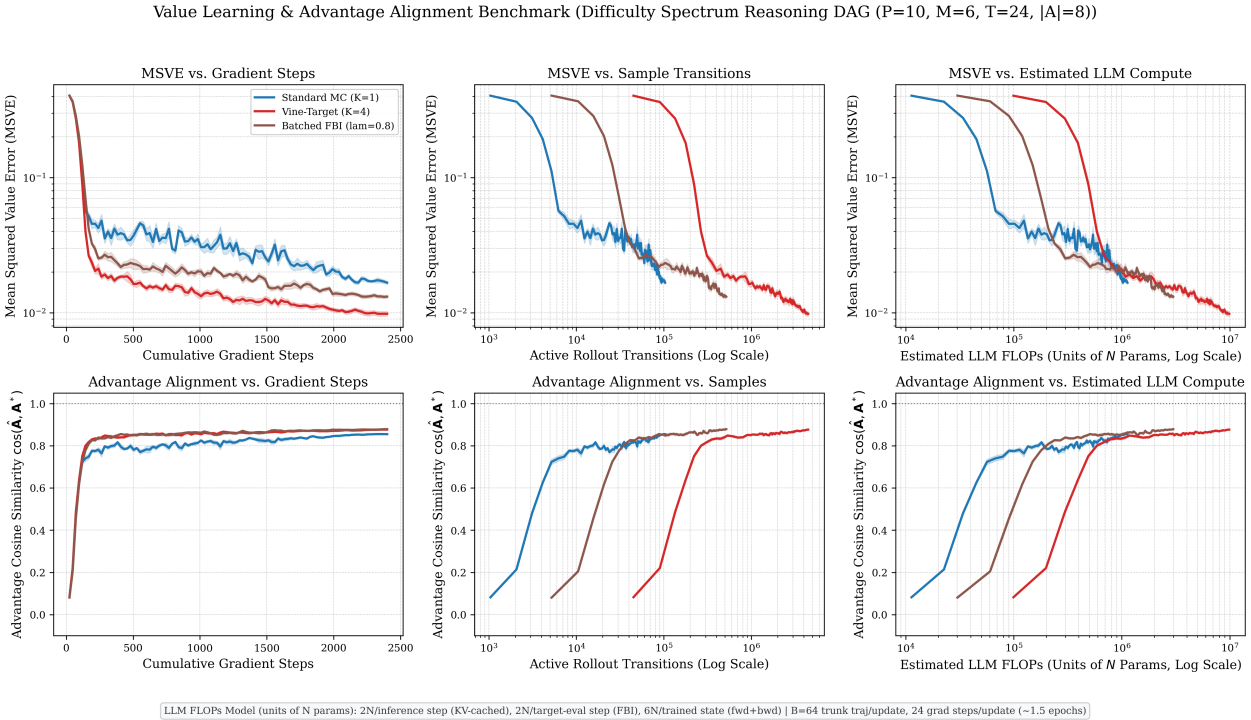


Figure 10: **Head-to-Head Learning Curves on Reasoning DAG.** Standard MC ($K = 1$), Vine-Target ($K = 4$), and Batched FBI ($\lambda = 0.8$). Notice that Batched FBI tracks Vine-Target's advantage alignment throughout training while using a fraction of the compute.

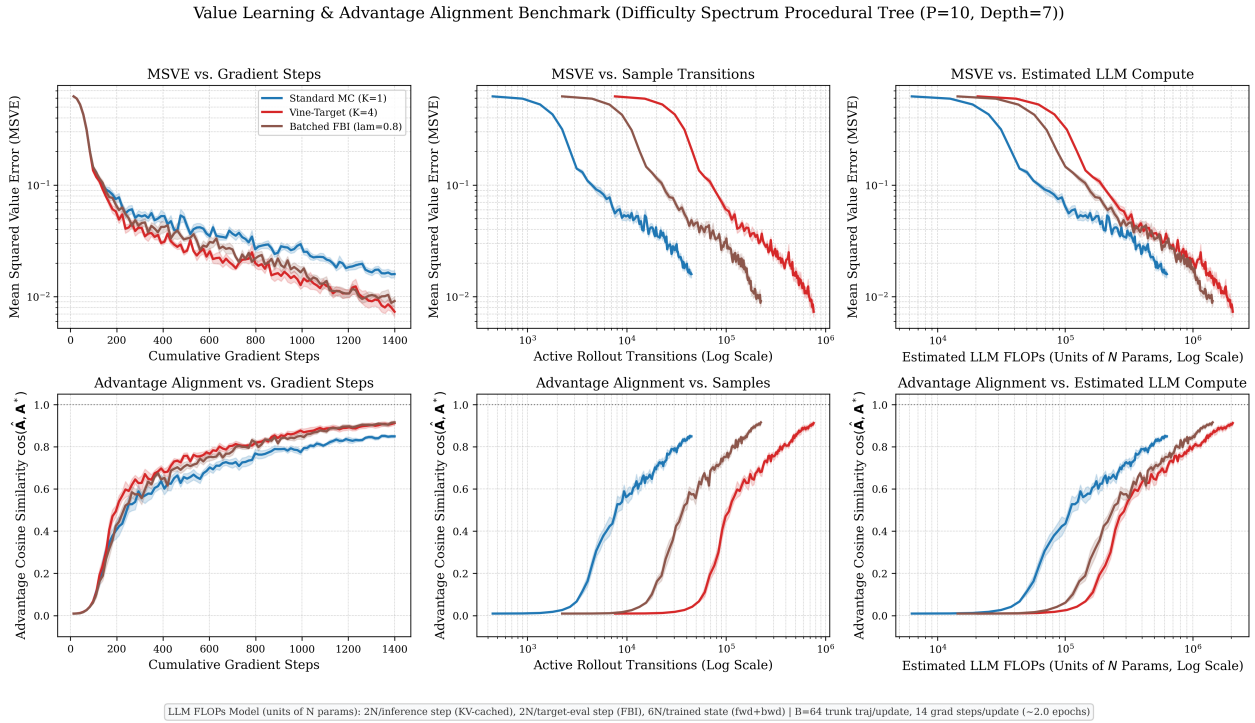


Figure 11: **Head-to-Head Learning Curves on Procedural Tree.**

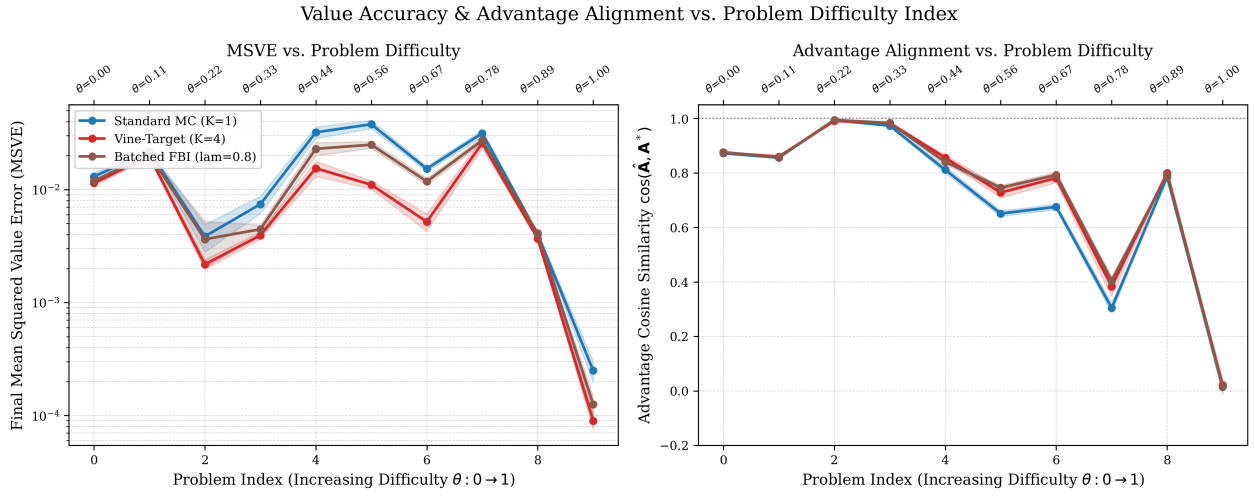


Figure 12: **Head-to-Head Performance Across Problem Difficulties on DAG.** Batched FBI ($\lambda = 0.8$) outperforms both MC and Vine ($K = 4$) on difficult problems ($\theta = 0.556, 0.667, 0.778$). All algorithms collapse at $\theta = 1.0$ due to extreme reward sparsity (Pass@1 $< 0.02\%$), demonstrating that no critic can learn when positive reward is never encountered.

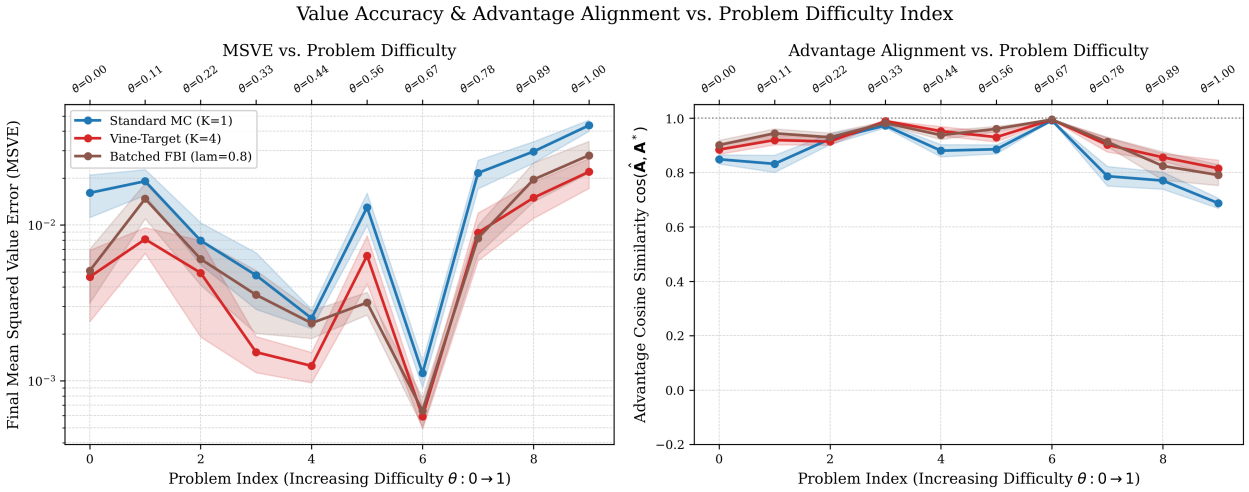


Figure 13: **Head-to-Head Performance Across Problem Difficulties on Tree.**

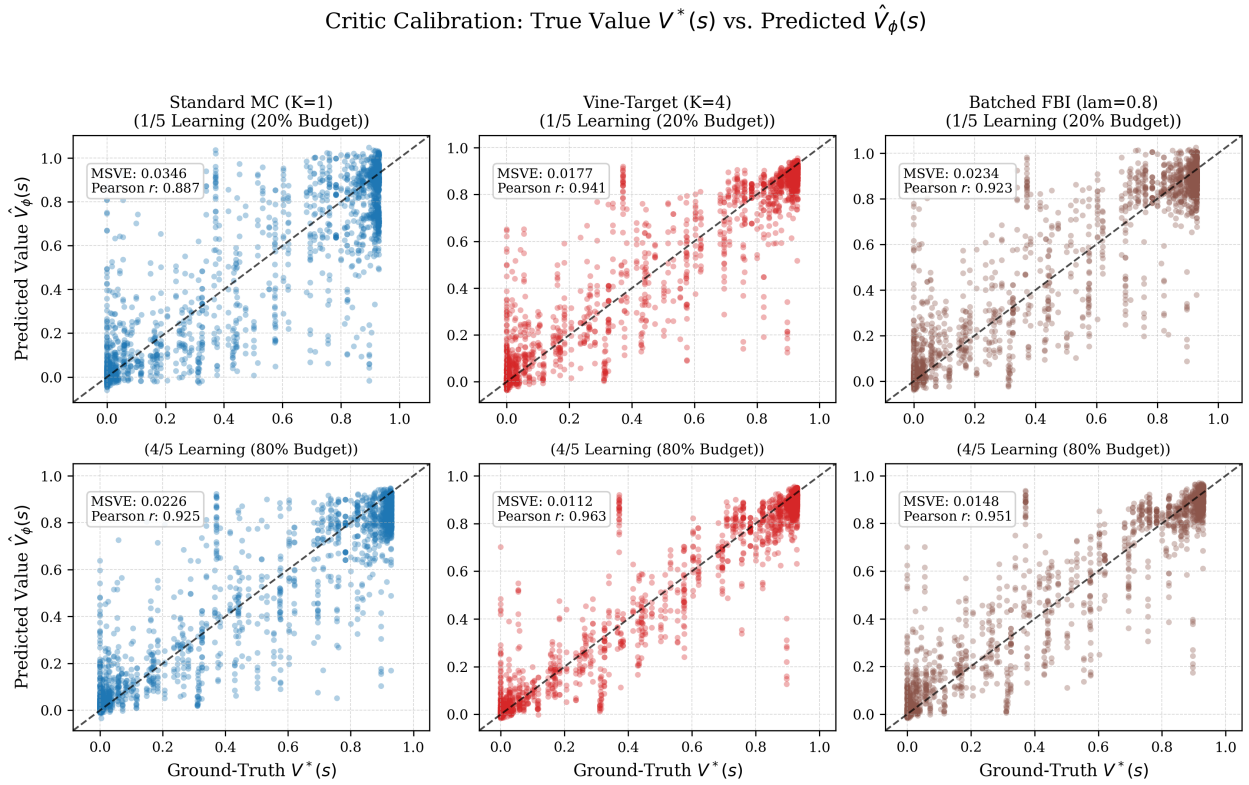


Figure 14: **Head-to-Head Critic Calibration on DAG:** Snapshots at 20% and 80% of training budget.

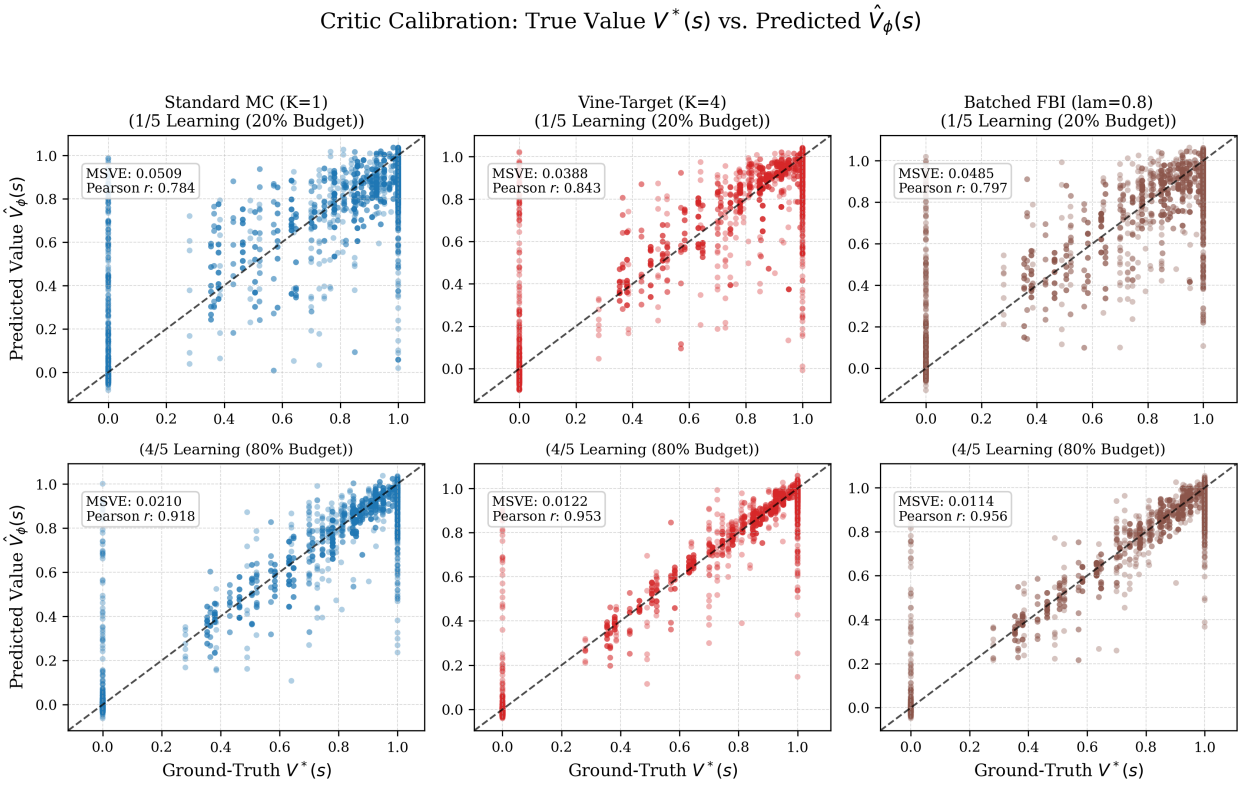


Figure 15: **Head-to-Head Critic Calibration on Tree:** Snapshots at 20% and 80% of training budget.

6 Appendix: Analysis of Staged FBI vs. Batched FBI

In this section, we analyze the theoretical motivation, algorithmic implementation, and empirical dynamics of **Staged Fitted Backward Induction ($\mathbf{FBI_{staged}}(\lambda)$)**, and explain why Batched FBI with higher λ ultimately proved superior.

6.1 Theoretical Motivation for Staged Fitting

In classical dynamic programming, backward induction solves a finite-horizon MDP by resolving values in strict reverse time: $t = T-1 \to T-2 \cdots \to 0$. In Batched FBI with pure 1-step targets ($\lambda = 0$), the target for step t queries the pre-update critic:

$$y_t \;=\; \frac{1}{K}\sum_{k=1}^{K}\Big(r_t^{(k)} + \gamma\, V_{\phi_{\text{old}}}\big(s_{t+1}^{(k)}\big)\Big).$$

Early in training, before the critic has learned values at step $t+1$, querying $V_{\phi_{\text{old}}}(s_{t+1})$ returns near-zero, uncalibrated estimates. Consequently, terminal reward information at $t = T$ propagates backward by at most 1 time step per outer update, requiring at least T outer iterations for reward signal to reach the root s_0.

Staged FBI was designed to eliminate this propagation delay within a *single* outer iteration by fitting the network in reverse backward-induction order (Figure 16):

1. **Base Stage at Terminal Step ($t = T-1$):** At the penultimate step, all K 1-step branches land on terminal states $s_T^{(1..K)}$ where exact environment rewards $R(s_T^{(k)}) \in \{0, 1\}$ are observed. The target $y_{T-1} = \frac{1}{K}\sum_{k=1}^{K} R(s_T^{(k)})$ requires no bootstrapping. The critic is trained for M gradient steps on $\mathcal{D}_{T-1} = \{(s_{T-1}, y_{T-1}), (s_T, y_T)\}$ to produce calibrated terminal weights $\phi^{(T-1)}$.
2. **Backward Induction Loop ($t = T-2$ down to 0):** Sweeping backward step-by-step:
   - Compute the target at step t using the **just-calibrated weights** $\phi^{(t+1)}$:

$$y_t \;=\; \frac{1}{K}\sum_{k=1}^{K}\Big(r_t^{(k)} + \gamma\, V_{\phi^{(t+1)}}\big(s_{t+1}^{(k)}\big)\Big) \;+\; \gamma\lambda\, \hat{A}_{t+1}^{(\lambda)}.$$

   - Add (s_t, y_t) to the cumulative active resolved set: $\mathcal{D}_t = \mathcal{D}_{t+1} \cup \{(s_t, y_t)\} = \{(s_\tau, y_\tau)\}_{\tau=t}^{T}$. Cumulatively retaining resolved steps prevents the shared neural network from catastrophically forgetting deeper steps $t+1 \ldots T$.
   - Take M gradient steps on $\mathcal{D}_t$ to obtain $\phi^{(t)}$, then proceed backward to $t-1$.

6.2 Why Staging Helps for Low $\lambda = 0.0$, but Higher $\lambda = 0.8$ Worked Best Anyway

Table 3 presents the empirical comparison of all four FBI variants across the DAG and Tree benchmark suites.

Environment	Algorithm	Final MSVE	Final AdvCos	Rollout Samples	FLOPs	Adv AUC
DAG Full	Batched FBI ($\lambda = 0.0$)	$1.50 \times 10^{-2} \pm 2.9 \times 10^{-4}$	$\mathbf{0.8898 \pm 0.002}$	518,417	3.00M	0.8280
($T = 24, P = 10$)	**Batched FBI ($\lambda = 0.8$)**	$\mathbf{1.32 \times 10^{-2} \pm 3.5 \times 10^{-4}}$	0.8789 ± 0.002	518,417	3.00M	**0.8426**
	Staged FBI ($\lambda = 0.0$)	$1.89 \times 10^{-2} \pm 5.0 \times 10^{-4}$	0.8755 ± 0.001	517,882	2.99M	0.8107
	Staged FBI ($\lambda = 0.8$)	$1.64 \times 10^{-2} \pm 4.3 \times 10^{-4}$	0.8684 ± 0.003	517,882	2.99M	0.8158
Tree Full	Batched FBI ($\lambda = 0.0$)	$1.35 \times 10^{-2} \pm 2.0 \times 10^{-3}$	0.8604 ± 0.022	224,000	1.43M	0.5570
($D = 7, P = 10$)	**Batched FBI ($\lambda = 0.8$)**	$\mathbf{9.15 \times 10^{-3} \pm 9.0 \times 10^{-4}}$	$\mathbf{0.9152 \pm 0.007}$	224,000	1.43M	**0.6938**
	Staged FBI ($\lambda = 0.0$)	$1.04 \times 10^{-2} \pm 1.5 \times 10^{-3}$	0.9028 ± 0.013	224,000	1.43M	0.6360
	Staged FBI ($\lambda = 0.8$)	$1.12 \times 10^{-2} \pm 1.7 \times 10^{-3}$	0.8767 ± 0.014	224,000	1.43M	0.6598

Table 3: **Comparison of Four FBI Variants (Batched vs. Staged, $\lambda = 0.0$ vs. $\lambda = 0.8$).**

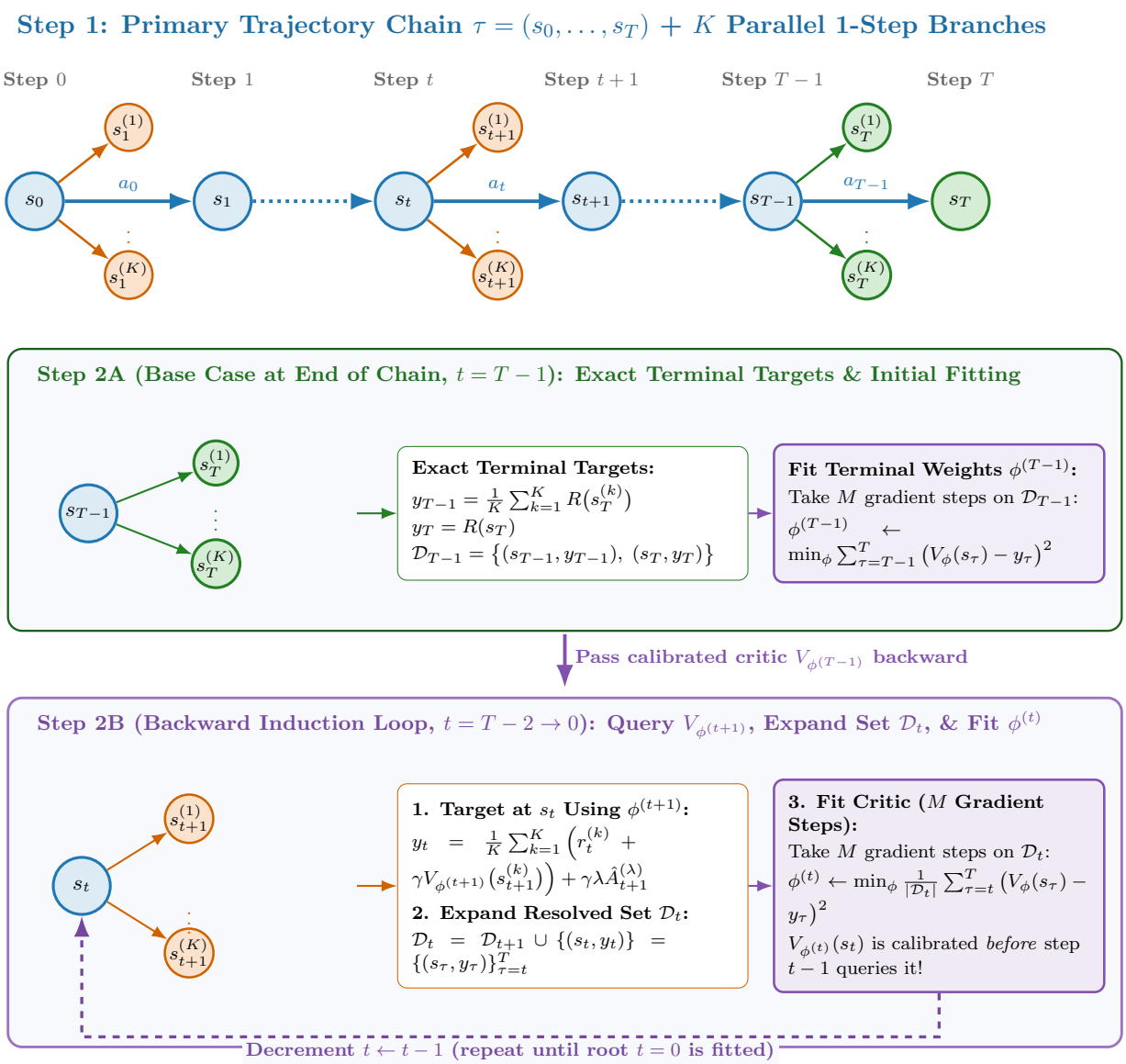


Figure 16: **Architecture of Staged Fitted Backward Induction ($\text{FBI}_{\text{staged}}(\lambda)$).** Reverse-time backward induction sweeps from $t = T-1$ down to 0, fitting $\phi^{(t+1)}$ before querying it to construct targets at step t.

1. Why Staging Helps for Low $\lambda = 0.0$ Early in Training: When $\lambda = 0$, the algorithm relies exclusively on 1-step Bellman bootstrapping. On the binary tree (`tree_full`), Staged FBI ($\lambda = 0.0$) significantly outperforms Batched FBI ($\lambda = 0.0$):

- Adv AUC is **0.6360** for Staged vs. only **0.5570** for Batched (+14.2% improvement).
- Final MSVE is 1.04×10^{-2} for Staged vs. 1.35×10^{-2} for Batched.

Because terminal reward is observed at depth $D = 7$, Batched FBI ($\lambda = 0.0$) requires 7 full outer updates for reward information to bootstrap back to the root s_0. Staged FBI propagates this reward all the way to s_0 in update 1, producing far superior early calibration.

2. Why Higher $\lambda = 0.8$ Worked Best Anyway (Making Staging Unnecessary): However, when λ is increased to 0.8, the multi-step trajectory accumulator $\hat{A}_t^{(\lambda)}$ carries terminal reward signal directly up the primary chain without depending on recursive 1-step bootstrap passes.

- On Tree Full, increasing $\lambda \to 0.8$ elevates Batched FBI's Adv AUC from 0.5570 to **0.6938** (+24.6%), completely eliminating the bootstrap lag.

- Under $\lambda = 0.8$, **Batched FBI strictly outperforms Staged FBI** on both DAG (1.32×10^{-2} vs. 1.64×10^{-2} MSVE) and Tree (9.15×10^{-3} vs. 1.12×10^{-2} MSVE).

3. The Neural Optimization Flaw of Staged FBI: Why does Batched FBI beat Staged FBI when $\lambda = 0.8$? In Staged FBI, the active training set $\mathcal{D}_t = [t \dots T]$ is inherently **bottom-heavy**: late transitions ($\tau \approx T$) remain in $\mathcal{D}_t$ across all T backward stages, receiving $T \times M$ gradient updates, whereas the root state s_0 enters only in the very last stage and receives only M updates. Furthermore, updating ϕ sequentially breaks the i.i.d. shuffling assumption of stochastic gradient descent.

In contrast, **Batched FBI shuffles all transition tuples across the entire rollout buffer for multiple global epochs**, ensuring that early, middle, and late reasoning tokens receive balanced gradient updates and learn shared causal representations simultaneously. Therefore, the combination of **Batched updates + $\lambda = 0.8$** delivers the optimal balance of fast reward propagation, low target variance, and balanced neural representation learning.

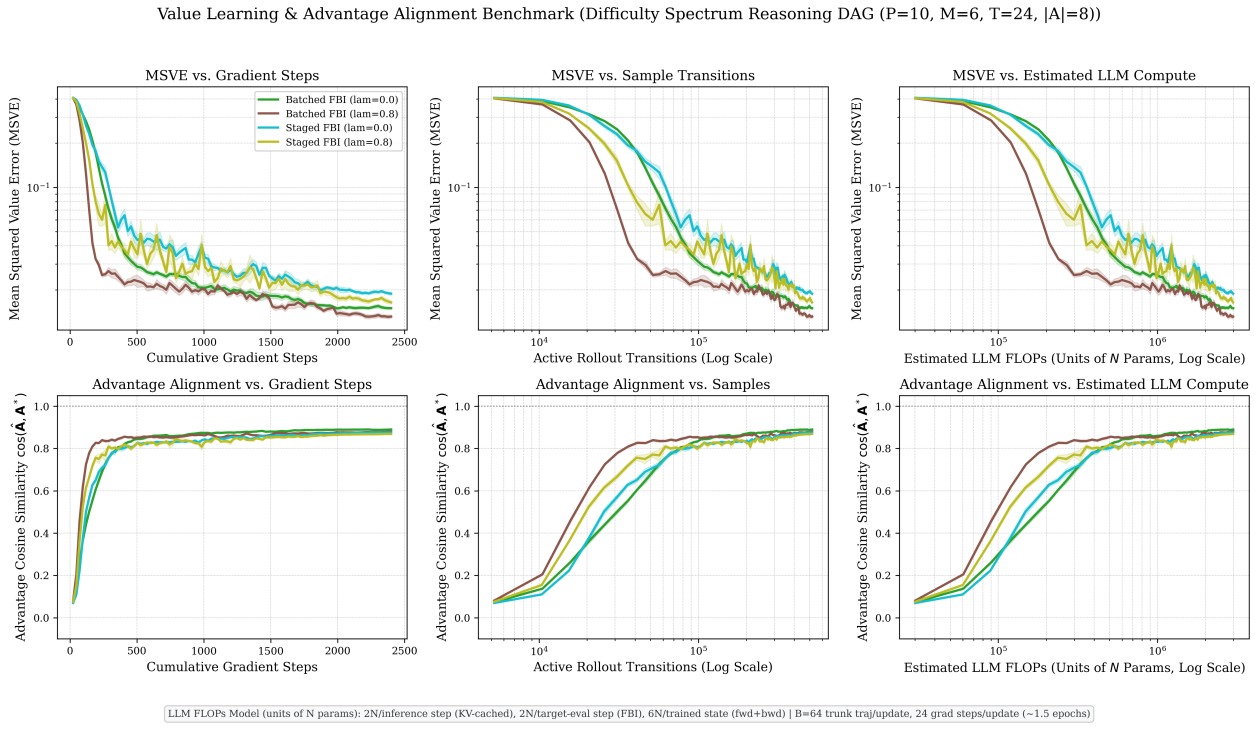


Figure 17: **Reasoning DAG: Four FBI Variants (Batched vs. Staged, $\lambda = 0.0$ vs. $\lambda = 0.8$).**

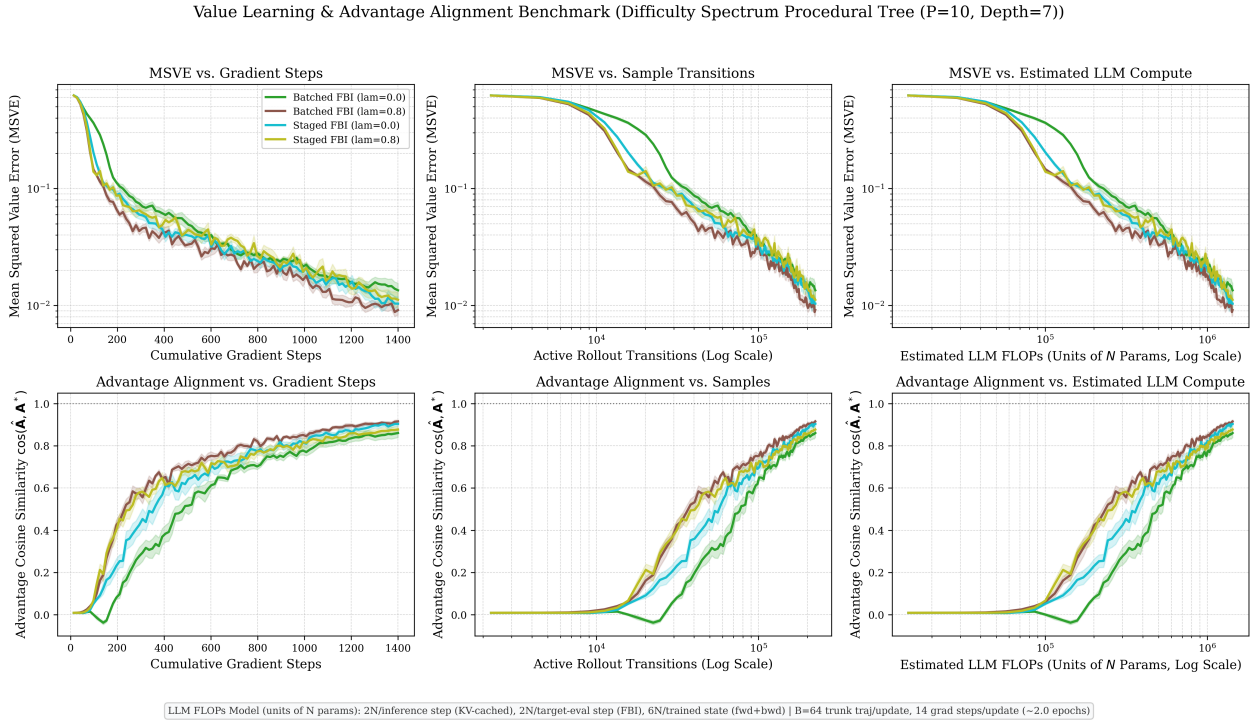


Figure 18: **Procedural Tree: Four FBI Variants (Batched vs. Staged, $\lambda = 0.0$ vs. $\lambda = 0.8$).**

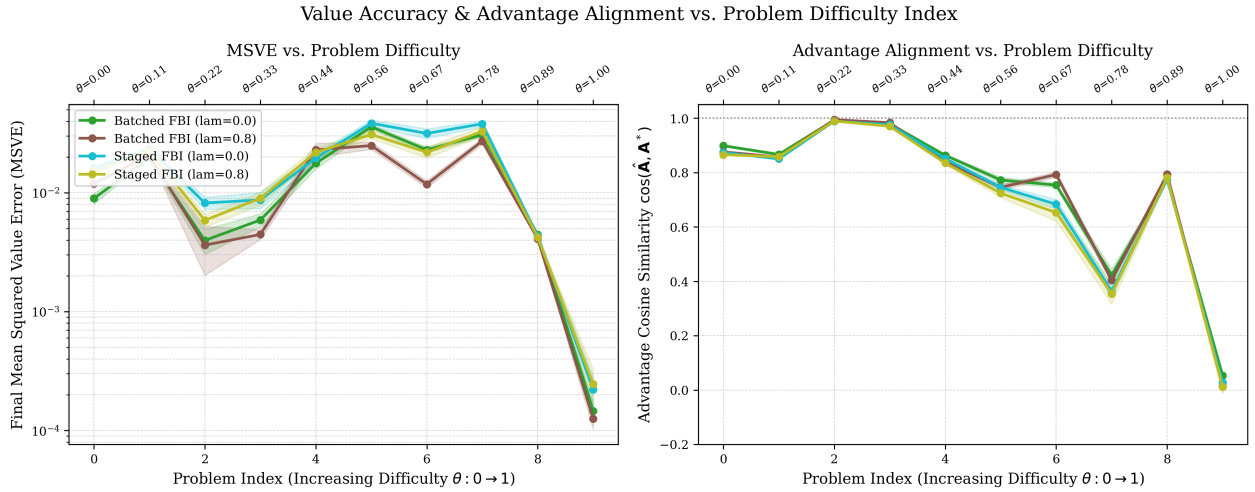


Figure 19: **Reasoning DAG: Four FBI Variants vs. Problem Difficulty $\theta \in [0, 1]$.**

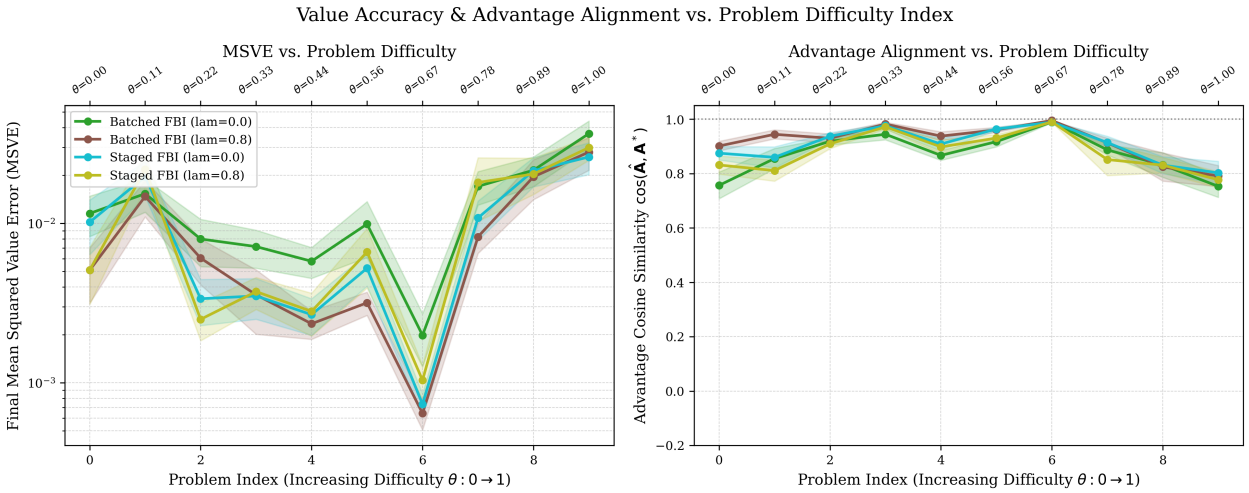


Figure 20: **Procedural Tree: Four FBI Variants vs. Problem Difficulty $\theta \in [0, 1]$.**

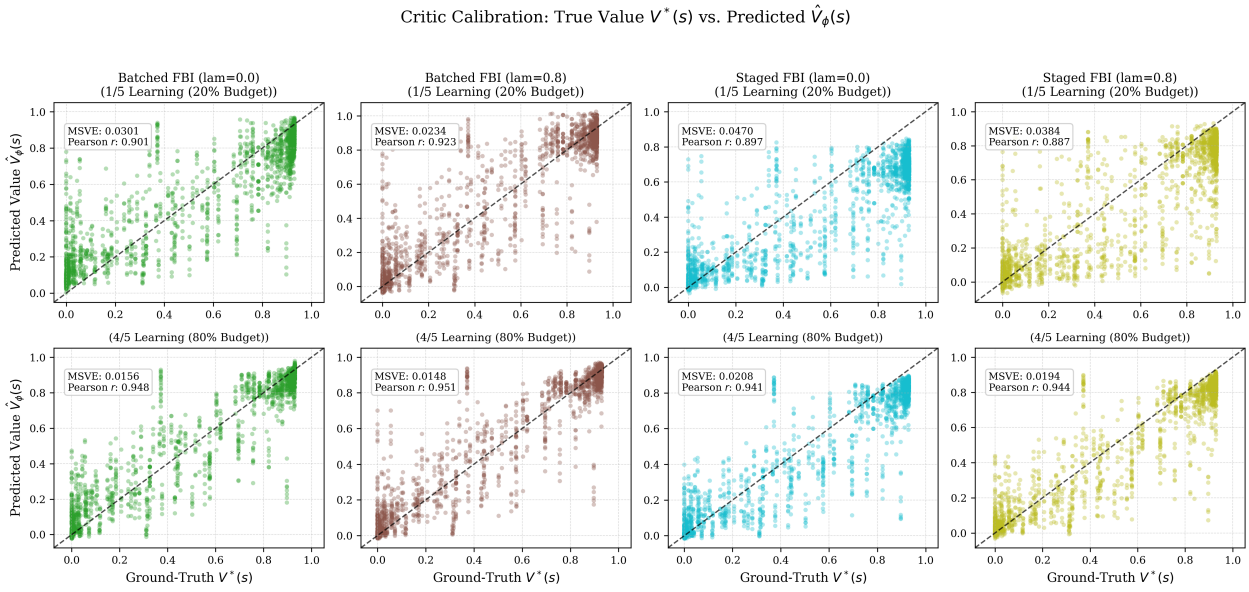


Figure 21: **Critic Calibration on DAG: Four FBI Variants.**

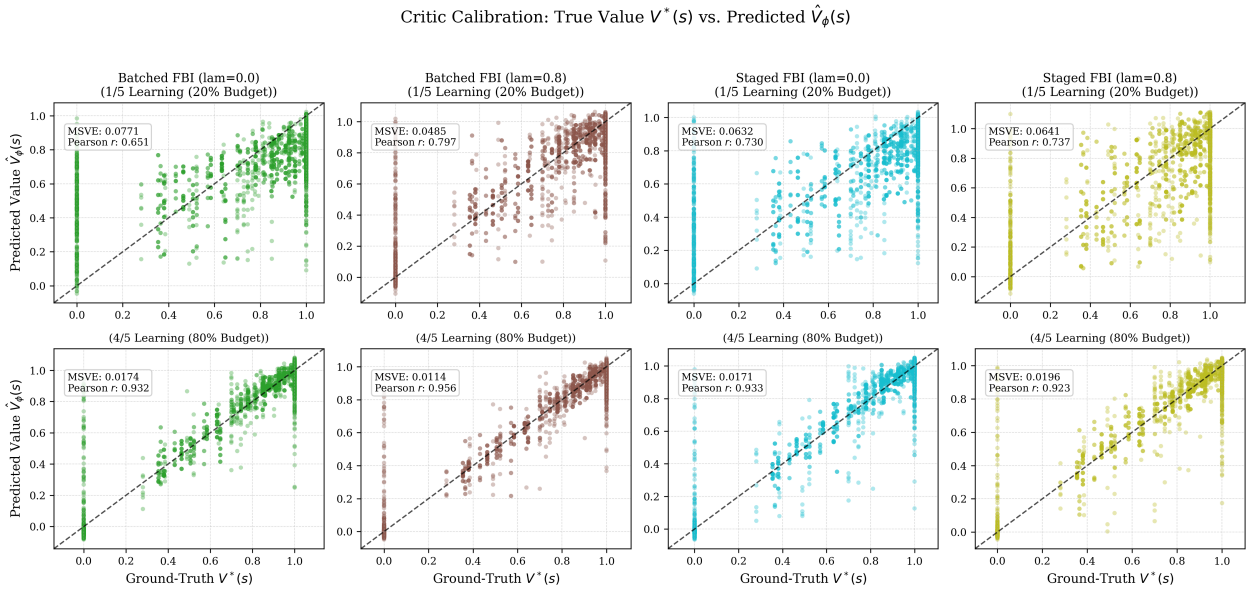


Figure 22: **Critic Calibration on Tree: Four FBI Variants.**